

In one line: Scope is **lexical**: fixed by where code is *written*. Every function keeps `[[Environment]]`, the Environment Record it was created in; a closure is that link keeping **bindings** (not values) alive. this is **dynamic**: set by *how the function is called* and looked up in the nearest record that has one — arrows have none, so they see the enclosing this.

1 closure + lexical this

```
function outer() { let x = 2; return () => this.n + x; }
const f = outer.call({ n: 1 }); f(); // 3
```

`outer()` returned: its context is popped, its record lives on

`x: f()` rec → `outer()` rec ✓
`this: f()` rec is lexical, skip → `outer()`'s `[[ThisValue]]`

Global record · `outer`: null
`var/function` → `globalThis`
`let/const/class`: own part
`[[OuterEnv]]` ↑
outer() Function record
`x: 2 this: {n:1}`
`outlives its call`
`[[OuterEnv]]` ↑
f() Function record
`this status: lexical`

2 loop bindings + closures

```
for (var i=0; i<3; i++) setTimeout(() => log(i));
```

`c0` `c1` `c2`
`ONE i (function scope) = 3`

`let i`: a new record per iteration, the value copied in before the 1st test, then after each body, before `i++`

`c0: i=0` `c1: i=1` `c2: i=2`

`for (const i=0; i<3; i++)`: no copy → `TypeError` at `i++`

3 what is this? first match

arrow function → the enclosing `this`; call / apply / bind cannot change it
`new f()` → the new object; derived class: none until `super()`
`g = f.bind(t); g()` → `t`, whatever call / apply pass; `new g()` ignores it
`f.call(t), f.apply(t)` → `t` as given; sloppy: null → `globalThis`, primitives boxed
`o.f()` `o[k]()` `o.f?.()` → `o`, the base — even when `f` lives on the prototype
plain `f()`, callbacks → strict: undefined; sloppy: `globalThis` of `f`'s realm
tagged template `o.f`x`` and with `(o) f()` → `o` too

Environment Records — the scope chain

- A record maps names → bindings; `[[OuterEnv]]` links it to the enclosing one. Lookup walks that chain; a name found nowhere → `ReferenceError` on read, "undefined" from `typeof`; a new global property on sloppy assignment.
- Declarative** (block, catch, `let` `const` `class`) and its two kinds: **Function** (+ `[[ThisValue]]`, `[[ThisBindingStatus]]`) lexical / initialized / uninitialized, `[[NewTarget]]` and **Module** (imports as live indirect bindings; `this` = undefined). **Object** (with). **Global** = an Object record over the global object (`var`, `function`) + a Declarative record (`let` `const` `class`) + `[[GlobalThisValue]]`.
- One **execution context** per script, module or call: code state, Function, Realm, ScriptOrModule, LexicalEnvironment, VariableEnvironment, PrivateEnvironment (`#names`, ES2022), Generator. `this` is *not* a field of it (ES5's `ThisBinding` was): `ResolveThisBinding` walks outward to the first record that has one.
- A **function object** carries `[[Environment]]` (the closure), `[[ThisMode]]` (lexical = arrow, strict, global = sloppy), `[[HomeObject]]` (for `super`), `[[Realm]]`.

Hoisting: bindings exist at entry

declaration	at scope entry	read before its line	scope
<code>var x</code>	undefined	undefined	function
<code>function f(){}</code>	the function	callable	function ^a
<code>let, const</code>	uninitialised	<code>ReferenceError</code> (TDZ)	block
<code>class C</code>	uninitialised	<code>ReferenceError</code> (TDZ)	block
<code>using r</code>	uninitialised	<code>ReferenceError</code> (TDZ)	block; disposed at exit
<code>var f = ()=>{}</code>	undefined	<code>f(): TypeError</code>	function
parameters	uninitialised ^b	(<code>a = b</code> , <code>b</code>): <code>ReferenceError</code>	own; body vars split off if any default
<code>import {x}</code>	linked first	live, read-only	module

^a in a block: block-scoped in strict code and modules; sloppy (Annex B) also makes a function-level var, undefined until the block runs (Chrome, Firefox; Safari differs). ^b undefined only with duplicate names (sloppy).

- TDZ** is *temporal*: from scope entry until the declaration *executes*. A closure called after `let x = 1` ran may read `x` though written above it. `typeof x` in the TDZ throws; `let x = x` throws; all cases of a switch share one block.
- Redeclaring**: `var + var` / `function` is allowed (`var a = 1; function a(){} → a is 1`); `let const class import` + anything in the same scope is an early `SyntaxError` — nothing runs. Classic scripts share one global lexical scope: `let x` in two `<script>`s throws in the second.
- Script top level: `var` / `function` become **non-configurable** `globalThis` properties (delete fails; sloppy `x = 1` makes a configurable one; `let const class` make none. A module (or CommonJS file) keeps its top level to itself.

Closures — what is kept

- The **binding**, shared: a later write is seen by every closure over it. To fix a value, pass it as an argument or copy it into a `const` inside the loop body. React's stale closure is a reader's old bindings.
- `for-of` / `for-in` with `let/const`: a fresh record each turn. `for(let;;)`: copied per iteration (drawing 2). `var`: one binding.
- Default parameters** get their own scope: closures in the parameter list never see the body's vars.
- Named function expression `function fact(){}`: `fact` is an **immutable** binding visible only inside; `fact = 1` is ignored (sloppy) or a `TypeError` (strict).
- Dynamic code**: direct `eval(s)` sees the local scope (sloppy: its vars land in the caller); indirect (`0`, `eval(s)` and new `Function(s)`) see **only the global** scope.
- V8**: variables an inner function references move into a heap **context**; every closure made in that scope points at that one context, so a big array captured by one closure lives as long as *any* of them. Leaks are reachability: a listener, timer or cache holding a closure.

Traps

- ! "let/const aren't hoisted" — they are, uninitialised (TDZ).
- ! "A closure copies the value" — it keeps the variable (3 3 3).
- ! An arrow as an object method: `{ n: 1, m: () => this.n }` reads the outer `this`.

Trace — each example run on its own

```
var a = 1; function f() { log(a); var a = 2; } f(); // undefined
{ log(b); let b = 2; } // ReferenceError
log(typeof h); var h = 1; function h() {} // "function"
var x = 1; function g(p = () => x) { var x = 2; return p(); }
g(); // 1: the parameter scope
(function k() { k = 1; return typeof k; })(); // "function"
// (strict code: TypeError)

const o = { n: 'o', m() { return this.n; },
  later() { setTimeout(() => log(this.n)); } };
const m = o.m; m(); // strict TypeError, sloppy globalThis.n
(o.m)(); o.m?().(); o['m'](); // 'o' 'o' 'o': reference kept
(0, o.m)(); (o.m = o.m)(); // a bare value, like m()
o.later(); // 'o': arrow reads later's this
o.m.bind({ n: 'x' }).call(o); // 'x': bind is permanent
function F() { log(this.n); }
new (F.bind({ n: 'x' })); // undefined: new skips boundThis
```

this — the details

- Arrows**: no own `this`, arguments, `super`, new, `target`; no prototype; new → `TypeError`; call / apply / bind ignore the `thisArg` (bind still pre-fills arguments). An arrow *property* of an object literal sees the `this` outside the literal.
- Lost this**: `arr.forEach(o.m)`, `const {m} = o`, `setTimeout(o.m)` hand over a value, not a reference. Fixes: `o.m.bind(o)`, `() => o.m()`, an arrow class field `onTap = () => this.go()` — fixed for life, but one closure per instance, not on the prototype.
- bind** is permanent: call, apply and a second bind cannot change it; new ignores the bound `this` but keeps the bound arguments.
- Classes** (always strict): instance field initialisers → the instance; static fields, static blocks, static methods → the class. Derived constructor: `this` before `super()` → `ReferenceError`, so is a second `super()`.
- `super` is fixed by `[[HomeObject]]`, where the method is written: moving a method changes its `this`, never its `super`. Getters and setters get the object accessed *through*, also via the prototype chain.
- Hosts pick it for callbacks**: array iteration methods take a `thisArg` (else undefined); Promise reactions undefined; DOM listeners `currentTarget`; browser timers window, even for a strict function.

top-level code	this
classic script, strict or not	<code>globalThis</code> (ES2020; page window, worker <code>self</code>)
ES module	undefined
CommonJS file (Node)	<code>module.exports</code>
direct / indirect eval	the caller's <code>this</code> / <code>globalThis</code>

Strict mode (classes, modules: always)

'use strict' first in a script or function body. `this` never substituted or boxed · undeclared assignment → `ReferenceError` · writing a read-only, getter-only or non-extensible property, delete of a non-configurable one → `TypeError` · with, delete `x`, duplicate parameters, `010`, `"\045"`, `eval/arguments` as names → `SyntaxError` · arguments no longer aliases parameters (also unaliased whenever parameters are not simple); arguments.caller → `TypeError` · eval's vars stay inside it · 'use strict' in a function with default, rest or destructured parameters → `SyntaxError`.

using (ES2027)

using `r = open()`: `const`-like, block-scoped, TDZ. At block exit calls `r[Symbol.dispose]()`, last declared first; null/undefined allowed, any other value without the method → `TypeError`; errors chain into a `SuppressedError`. Allowed in blocks, function bodies, static blocks, a module's top level, `for` / `for-of` heads; not at a classic script's top level, not directly in a switch, no destructuring. `await` using awaits `[Symbol.asyncDispose]()`, falling back to `[Symbol.dispose]()`.